Remote Internship Program

Task 1 Report

Basic Network Sniffer Using Python and Scapy

Bouhali Douâa

April 28, 2026

1 Objective

The goal of this task was to build a basic network packet sniffer using Python. The sniffer captures live network traffic, extracts key information from each packet (such as source/destination IP addresses and protocol types), and displays raw payload data where available. This exercise develops foundational skills in network analysis and low-level traffic inspection.

2 Tools and Environment

Component	Details
Language	Python 3
Library	Scapy
Platform	Windows Visual Studio Code

Scapy was installed via pip:

```
pip install scapy
```

Packet sniffing requires elevated privileges. On Linux this means running as `root`; on Windows, the terminal must be launched as Administrator.

3 Background

Network packets are structured in layers. A typical packet follows the hierarchy:

`[Ethernet]` → `[IP]` → `[TCP/UDP]` → `[Payload]`

Understanding this layered model is essential for extracting meaningful data. Each layer encapsulates the one above it, and tools like Scapy allow us to inspect each layer independently.

4 Implementation

4.1 Minimal Sniffer

The first version captures 10 packets and prints a one-line summary of each:

```
1 from scapy.all import sniff
2
3 def process_packet(packet):
4     print(packet.summary())
5
6 sniff(prn=process_packet, count=10)
```

The `sniff()` function is Scapy's core capture method. The `prn` argument specifies a callback function executed for every captured packet, and `count` limits the capture to a fixed number.

```
[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 35.186.224.24
Protocol: UDP
Payload: b'\xe5\x00\x00\x00\x01\x08\xfd{\x043M-\xdb<\x00@\x17\xa4\x8a\xbfX>3ms\xba\xe2\xd7D\xdc\x
e8\xb1\t\xfc\xa5S~\xb3\x8b\x99'

[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 35.186.224.24
Protocol: UDP
Payload: b'\xe6\x00\x00\x00\x01\x08\xfd{\x043M-\xdb<\x00@H8l\xa4\x06T<\t\xf1E3\x14\xf2"\x02\x1f\x
c7v\xa12=\x82\x9a \xa0X2\x87\xdai\xa0\xbf=\x01\x0b:R\xc9\x14\x11\xf9\x0f\xa2\x9b<\x97VZ4\xae]\xb1
\xf5\x8etE8\xc3\xca)\x1fi\xf9\xe4\xa6\xa3\xfd8WwL\xb1\xa2V\xfd{\x043M-\xdb<\x14\x0e\xaa]\xc74\xd4
\n+\xff\xebv\x19h-*\xb4v\x13\x02\xd8\xc5R\x97\x03\x15\x85\xc7o\xf7c\\\x08d\xbcd\x80*aS`U\xadh\xa8
\x85\xf1%\xa6\xe0K6fAL\xdb"\x05*X\x1f\xd82\xda\xb2Pq \x9e\xc1'
```

Figure 1: Basic sniffer results

4.2 Advanced Sniffer with Deep Inspection

The improved version extracts structured data from each packet:

```python
from scapy.all import sniff, IP, TCP, UDP, Raw

def process_packet(packet):
    if packet.haslayer(IP):
        ip_layer = packet[IP]

        print("\n[+] New Packet Captured")
        print(f"Source IP      : {ip_layer.src}")
        print(f"Destination IP : {ip_layer.dst}")

        if packet.haslayer(TCP):
            print("Protocol: TCP")
        elif packet.haslayer(UDP):
            print("Protocol: UDP")
        else:
            print("Protocol: Other")

        if packet.haslayer(Raw):
            print(f"Payload: {packet[Raw].load}")

sniff(prn=process_packet, store=False)
```

Key elements of this implementation:

- `haslayer(IP)` — checks whether the packet contains an IP layer before attempting extraction.
- `packet[IP].src / .dst` — retrieves the source and destination IP addresses.
- `haslayer(TCP/UDP)` — determines the transport protocol.
- `packet[Raw].load` — accesses the raw application-layer payload.
- `store=False` — prevents Scapy from storing packets in memory, reducing resource usage during long captures.

```
[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 35.186.224.24
Protocol: UDP
Payload: b'O\xfd{\x043M-\xdb<~\x1e\xf2\x10\x19\xc4\x9d\xc6\x9a~1\xde\x0e\xa0\xca\xf7^El\xf5\xfa!,
{fTi\xaa\x173\xfaC\x05g\xec\x00\xda\x08>]PmjnQ\xdfR\xcf\x06\x89$z\x1c\xc877\xac\x0c\x96\x08\x93B\
x03d;Gl\x96r\xa6\xd2\xdc\x92U\x17\x7f >?\x946\x102\xbd\x06\x10\x02\xe5\x8e\xe4y\xe1\xa7\xb4\xb0'

[+] New Packet Captured
Source IP: 35.186.224.24
Destination IP: 172.20.10.3
Protocol: UDP
Payload: b"H\xd2 \x98\xd0<\x1c\xbe\xdf\x86\xe3\x81\xcb\xfb'0\x93=\xd4\x84\xc4\x8e\xec"
```

Figure 2: Advanced sniffer results 1

```
[+] New Packet Captured
Source IP: 20.189.173.13
Destination IP: 172.20.10.3
Protocol: TCP

[+] New Packet Captured
Source IP: 35.186.224.24
Destination IP: 172.20.10.3
Protocol: UDP
Payload: b'^\xae\xe7:\xa6\xff\x15#\xa7bo\xd4\xfcZ\xdc\xcb\xf1\xb0qV\xe7\x99\x8c\xd6U\x81s[+\x9e\x
0ff\x15\xd4\xc3\x7f\x85\xbe\x135w\xbem\xdc\xb4\xfct\x02\x87Z\xf0\x8c4\xa6\xfd\x00p*mD,eN\x17 \xa4
\xa8r\xbc4\x06\x8b\x8aG\ry}\xd3Z>x\xcc\xd8k\x95#\x80\x0c\x06\xc7\xec\xb4\xa0\xf2\x9b\xc3]G\xbc\xa
c<\x8e\xd61L\xb8]\xba{\xeb@\x98\xc9\xcb\xa1\x14\xc6g^\x17\x8fJ\x10o\xf1\xf2\n\x85"\x8b}\x1f#0\xd7
\xfe\xb0$\x13\xa8\x06\x03\x15\xd5?\xa7\x0f\x9e\xfe^\xf24\xf3\xe7\xe3\x1a\x1c\xa0&\x8e'
```

Figure 3: Advanced sniffer results 2

```
[+] New Packet Captured
Source IP: 52.202.46.90
Destination IP: 172.20.10.3
Protocol: TCP

[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 3.160.196.61
Protocol: TCP
Payload: b'\x00'

[+] New Packet Captured
Source IP: 3.160.196.61
Destination IP: 172.20.10.3
Protocol: TCP

[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 185.201.2.39
Protocol: TCP

[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 185.201.2.39
Protocol: TCP
```

Figure 4: Advanced sniffer results 3

```
[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 8.8.8.8
Protocol: Other
Payload: b'abcdefghijklmnopqrstuvwabcdefghi'

[+] New Packet Captured
Source IP: 8.8.8.8
Destination IP: 172.20.10.3
Protocol: Other
Payload: b'abcdefghijklmnopqrstuvwabcdefghi'

[+] New Packet Captured
Source IP: 172.20.10.3
Destination IP: 8.8.8.8
Protocol: Other
Payload: b'abcdefghijklmnopqrstuvwabcdefghi'

[+] New Packet Captured
Source IP: 8.8.8.8
Destination IP: 172.20.10.3
Protocol: Other
Payload: b'abcdefghijklmnopqrstuvwabcdefghi'
```

Figure 5: Advanced sniffer results 4

5 Traffic Generation for Testing

To observe real results during the sniffing session, outbound traffic was generated using:

```
ping google.com
```

Additionally, browsing websites during the capture session produced HTTP/HTTPS TCP packets visible in the sniffer output.

6 Results

The sniffer successfully captured live packets and displayed structured output including source and destination IPs, protocol identification (TCP/UDP), and raw payload data for unencrypted traffic. ICMP packets from the ping command were also captured, confirming that the sniffer operates at the IP layer regardless of transport protocol.

7 Conclusion

This task provided practical experience with low-level network programming in Python. Using Scapy, a functional packet sniffer was built from scratch that can inspect traffic at the IP, TCP/UDP, and payload layers. This forms a strong foundation for more advanced tasks such as protocol filtering, traffic logging, and intrusion detection concepts.